Day 144 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 5 reference

Watch: Classifier-Free Guidance Diffusion Models for Image Synthesis (AI Focus) | Mastering Negative Prompts in Stable Diffusion (Making AI Magic)

`day144_prompt_engineering.py`, segment by segment, in the order `main()` uses things. The captioned guess, the training loop with dropped captions, guided sampling, the two measurements and digit naming are new; the networks (Day 126), scorer (Day 138), schedule (Day 140), model design (Day 141), averaged weights and DDIM (Day 142) and encoders and captions (Day 143) are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: c = caption embedding (64 numbers) · a, b = guesses with and without the prompt · w = guidance scale · A_t = ABAR[t]

Segment 0 — Setup

Imports: five earlier parts of this week

CODE

```
sys.path.insert(0, HERE)
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day139_gan_build_and_forward_diffusion as d139
    import day140_exercise as d140
    import day141_exercise as d141
    import day142_exercise as d142
    import day143_exercise as d143
```

- `import day143_exercise as d143` — importing an exercise runs its own checks once (here, a short training run); redirect_stdout hides what they print.
- `import day139_gan_build_and_forward_diffusion as d139` — a lesson imported as a module: its experiments sit under `if __name__ == "__main__"` and do not run.

Constants

CODE

```
ABAR = d140.ABAR
ACTS = ["relu", "relu", "linear"]
SIZES = [784 + 32 + 64, 512, 512, 784]
STEPS = 20000
DROP = 0.15
HOLD = (7, 1)
SAMPLE_STEPS = 100
W_DEFAULT = 2.0
N = 200
```

- `SIZES = [784 + 32 + 64, 512, 512, 784]` — the input width written as a sum, so its three parts stay visible.
- `DROP = 0.15` — a share, used as `rng.rand(batch) < DROP`: True for about 15% of rows.

Segment 1 — The new parts

guess_c: one more input

CODE

```
def guess_c(net, xt, t, c):
    A = d126.forward(net, np.concatenate([xt, d140.time_embed(t), c], 1).astype(np.float32), ACTS)
    return A, A[-1].astype(np.float64)
```

- `np.concatenate([xt, d140.time_embed(t), c], 1)` — three blocks of columns side by side: 784 + 32 + 64 = 880 numbers per row.
- `A[-1].astype(np.float64)` — the last layer's output, in double precision for the sampler's arithmetic.

train_cond: dropping captions

CODE

```
def train_cond(X2, y, thick, lean, txt, steps=STEPS, drop=DROP, hold=HOLD, seed=0, batch=128,
    lr=1e-3, every=2000):
    net = d126.init_net(SIZES, seed); net[-1] = (net[-1][0] * 0, net[-1][1])
    opt = d126.Adam(net, lr); ema = [(W.copy(), b.copy()) for W, b in net]
    rng = np.random.RandomState(seed); log = []; run = 0.0
    for s in range(1, steps + 1):
        i = rng.randint(0, len(X2), batch); x0 = X2[i].astype(np.float64); t = rng.randint(1,
            len(ABAR), batch)
        e = rng.standard_normal(x0.shape); xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 -
            ABAR[t])[:, None] * e
        c = d143.embed_texts(txt, [d143.caption(y[k], thick[k], lean[k], rng, hold) for k in i])
        c[rng.rand(batch) < drop] = 0.0
        A, x0h = guess_c(net, xt, t, c); miss = x0h - x0
        grads, _ = d126.backward(net, A, ACTS, (miss * 2 / miss.size).astype(np.float32))
        opt.step(net, grads); d142.ema_update(ema, net); run += (miss ** 2).mean()
        if s % every == 0:
            log.append((s, run / every)); run = 0.0
    return net, ema, log
```

- `net[-1] = (net[-1][0] * 0, net[-1][1])` — the output layer's weights replaced by zeros (Day 141's fix for a stalled start).
- `[d143.caption(y[k], thick[k], lean[k], rng, hold) for k in i]` — a new random caption for each of the 128 digits, every step.
- `c[rng.rand(batch) < drop] = 0.0` — a boolean mask picks rows; assigning 0.0 sets every number in those rows: 'no caption'.
- `d142.ema_update(ema, net)` — Day 142's averaged copy, changed in place after each step.
- `log.append((s, run / every)); run = 0.0` — the average loss over the last 2,000 steps, then the running sum starts again.

cfg_sample: two calls per step, one formula

MATH

$$guided\ x_0 = b + w\ (a - b)$$

CODE

```
def cfg_sample(net, cpos, n, w, cneg=None, steps=SAMPLE_STEPS, seed=1):
    cp = np.repeat(cpos[None], n, 0)
    cn = np.zeros_like(cp) if cneg is None else np.repeat(cneg[None], n, 0)
    def x0_fn(x, t):
        tt = np.full(len(x), t); a = guess_c(net, x, tt, cp)[1]
        if w == 1.0 and cneg is None:
            return a
        b = guess_c(net, x, tt, cn)[1]
        return b + w * (a - b)
    return d141.to_pixels(d142.ddim_sample(x0_fn, n, np.random.RandomState(seed), steps, 0.0))
```

- `np.repeat(cpos[None], n, 0)` — one embedding copied into n rows, one per sample.
- `np.zeros_like(cp) if cneg is None else ...` — the second guess uses 'no caption' unless a negative prompt is given.
- `def x0_fn(x, t):` — an inner function that remembers net, cp, cn and w; Day 142's ddim_sample calls it at every step.
- `if w == 1.0 and cneg is None: return a` — w = 1 makes b + w(a – b) equal a, so the second call is skipped.
- `return b + w * (a - b)` — the guided guess.

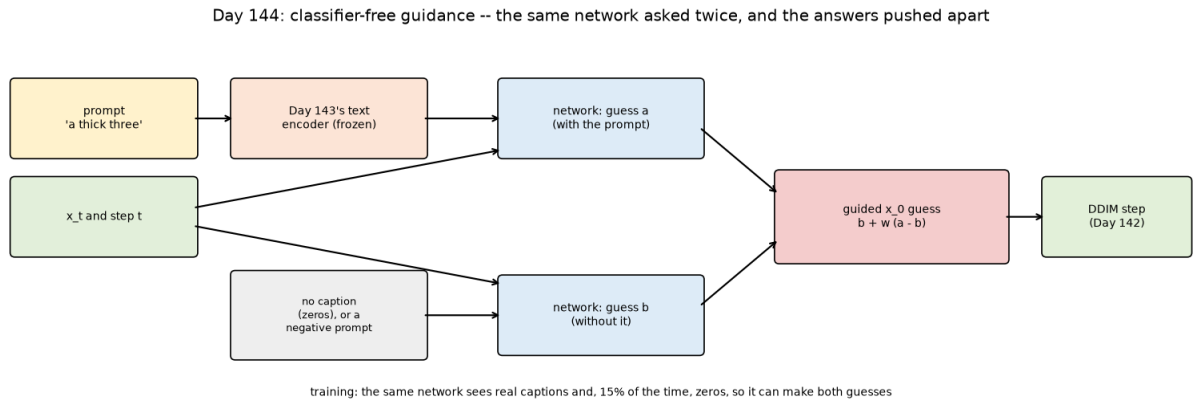
ink_lean and said

CODE

```
def ink_lean(F):
    R = np.repeat(np.arange(28.0), 28)[None]; Cc = np.tile(np.arange(28.0), 28)[None]
    m = F.sum(1, keepdims=True) + 1e-9; mr = (F * R).sum(1, keepdims=True) / m; mc = (F *
        Cc).sum(1, keepdims=True) / m
    return F.mean(1), -(F * (R - mr) * (Cc - mc)).sum(1) / (F * (R - mr) ** 2).sum(1)
def said(sc, F):
    return np.concatenate([sc.head.forward(sc.feats(F[i:i + 1000])).argmax(1) for i in range(0,
        len(F), 1000)])
```

- `return F.mean(1), -(...).sum(1) / (...).sum(1)` — two arrays returned as a tuple: thickness and lean for every image.
- `sc.head.forward(sc.feats(...)).argmax(1)` — the scorer's own feature network and head; argmax picks the digit with the largest score.
- `for i in range(0, len(F), 1000)` — in batches of 1,000, joined by np.concatenate.

SYSTEM DIAGRAM



Classifier-free guidance.

Segment 2 — The experiments

demo_scale: six scales, ten prompts each

CODE

```
yy = np.repeat(np.arange(10), N); out = []; keep = {}
for w in (0.0, 1.0, 2.0, 3.0, 5.0, 8.0):
    t0 = time.perf_counter(); F = np.vstack([cfg_sample(ema, d143.embed_texts(txt, [[word]])[0],
        N, w, seed=1 + k) for k, word in enumerate(d143.NUM)])
    took = time.perf_counter() - t0; i, fid, p, r, _ = sc(F); acc = float(np.mean(said(sc, F) ==
        yy))
    out.append((w, acc, fid, r)); keep[w] = F[::N // 4][:40]
    print(f"{w:5.1f} {acc:12.3f} {i:6.2f} {fid:8.2f} {p:10.3f} {r:7.3f} {d142.grey_share(F):11.4f}
        {F.mean():6.3f} {took:5.0f}s")
```

- `np.repeat(np.arange(10), N)` — 0 two hundred times, then 1, ...: the digit each of the 2,000 samples was asked to be.
- `d143.embed_texts(txt, [[word]])[0]` — a one-word caption: a list holding one list of words; [0] takes its single embedding.
- `seed=1 + k` — a different starting noise for each digit's 200 samples.
- `keep[w] = F[::N // 4][:40]` — every 50th sample: four of each digit, in order, for the figure.

run_prompt: one prompt, measured

CODE

```
def run_prompt(sc, txt, ema, prompt, w=W_DEFAULT, neg=None, want=None):
    F = cfg_sample(ema, d143.embed_texts(txt, [prompt.split()])[0], N, w, None if neg is None else
        d143.embed_texts(txt, [neg.split()])[0])
    lab = said(sc, F); ink, lean = ink_lean(F); cnt = np.bincount(lab, minlength=10); top =
        np.argsort(-cnt)[:2]
    right = f"{np.mean(lab == want):.3f}" if want is not None else "-"
    print(f"{repr(prompt):>30} {repr(neg) if neg else '-':>16} {right:>7} {ink.mean():7.4f}
        {lean.mean():7.4f} "
          f"{d142.grey_share(F):7.4f}   {d143.NUM[top[0]]} {cnt[top[0]]}, {d143.NUM[top[1]]}
              {cnt[top[1]]}")
    return F
```

- `None if neg is None else d143.embed_texts(...)[0]` — a conditional expression as an argument: the negative embedding only when one is given.
- `np.bincount(lab, minlength=10)` — how many samples were named as each digit.

- `np.argsort(-cnt)[:2]` — the two most frequent digits.
- `f"{np.mean(lab == want):.3f}" if want is not None else "-"` — a dash when there is no single intended digit (section 7).

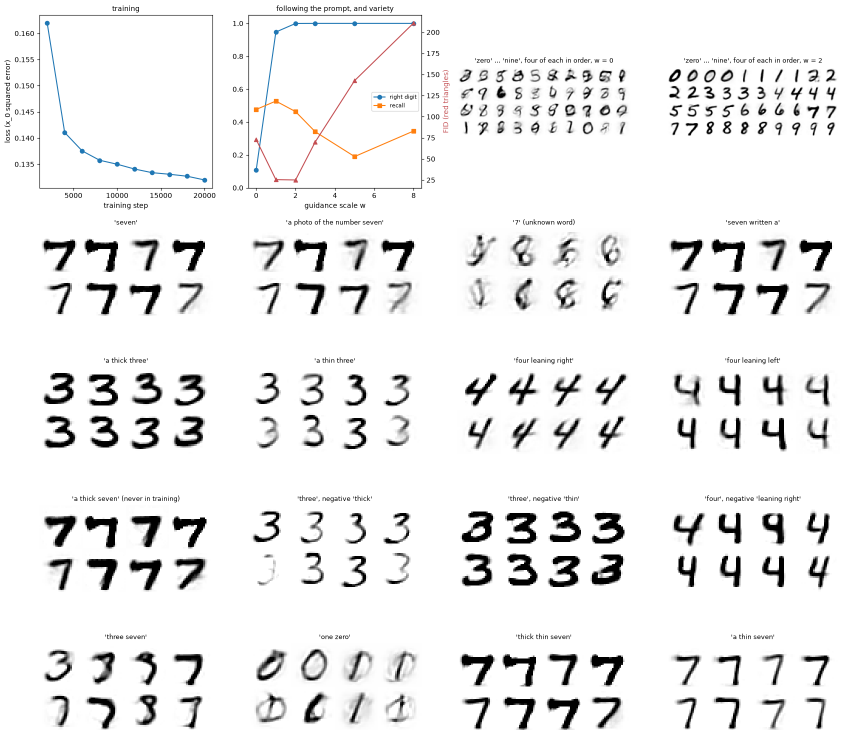
demo_paraphrase: an unknown word is an empty caption

CODE

```
e7, e0 = d143.embed_texts(txt, [["7"], []])
print(f"'7' against an empty caption: cosine {float(e7 @ e0):.6f}; an empty caption against 'no
    caption' (zeros, used in training): "
      f"{'different' if np.abs(e0).max() > 0 else 'the same'} (largest entry
          {np.abs(e0).max():.3f})")
```

- `[["7"], []]` — two captions: one holding the word '7', one empty; the bag of words drops '7', so both become all zeros.
- `e7, e0 = ...` — the two rows of the result unpacked into two names.
- `float(e7 @ e0)` — two 1-D arrays: the product is a single number, their cosine (both have length 1).

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.concatenate([a, b, c], 1)` — join blocks of columns.
- `A[mask] = 0.0` — set whole rows chosen by a boolean mask.
- `np.repeat(v[None], n, 0)` — n copies of one row.
- `def outer(): def inner(): ...` — a closure handed to another function.
- `x if cond else y (as an argument)` — a conditional expression.
- `np.argsort(-a)[:k]` — the positions of the k largest values.

End of Day 144 syntax reference. For the experiments and what they showed, see the Day 144 complete guide.